BugIt
by Taskivator

完整的分步操作手册

BugIt

VS Code 的 QA 缺陷提交 Agent

首次设置与日常使用：为从未做过类似操作的人编写的详细分步指南，逐步点击讲解。无需任何技术背景。按顺序操作，你今天就能提交规范的缺陷工单。

运行环境：VS Code。Windows（主要）、macOS 和 Linux

 © 2026 Taskivator. All Rights Reserved.

翻译说明。
本文档由机器翻译生成，未经母语者审校。以英文版本为准：如有出入，以英文文本为准。如需最准确、最新的表述，请参阅英文文档。

本指南内容

请按顺序完成一次**第一部分**。之后的内容你可以在需要时随时查阅。末尾的**故障排查**和**常见问题**解答了最常见的问题，请在发邮件寻求支持之前先查看这些内容。

第一部分 · 设置（只需完成一次）

步骤 0：选择你的 AI 助手

步骤 1：安装 VS Code

步骤 2：解压你下载的 BugIt

步骤 3：在 VS Code 中打开 BugIt

步骤 4：在聊天中选择 BugIt 代理

步骤 5：激活你的许可证

步骤 6：接受条款（仅需一次）

步骤 7：运行 "Begin setup"

步骤 8：连接你的缺陷跟踪系统

步骤 9：保存你的登录信息

步骤 10：确认你已就绪

第二部分 · 日常使用

命令速查

第三部分 · 打造专属设置

第四部分 · 更新、备份与安全

第五部分 · 故障排查与常见问题

常见问题

Team 许可证：常见问题

第六部分 · 许可证与支持

让你安全无虞的唯一规则
BugIt **绝不会**在向你展示预览之前提交任何内容。不可逆的操作需要你输入 `FILE IT`，仅输入"是"不会执行。你始终掌控全局。想零风险练习一下？输入 `dry run`，它会写出完整的报告，但**不会**提交。

首次设置 vs. 日常使用
你只需完整走一遍**第一部分**一次。之后，打开 BugIt 会很快，更新也只需一条命令（`Update`）。详见第四部分。

第一部分 · 设置：只需完成一次

大约需要 15 分钟。请按顺序完成各步骤，每一步都建立在上一步的基础上。不要跳步。

0 选择你的 AI 助手

> **本指南采用 GitHub Copilot 的方式**
>
> BugIt 是"驾驶员"，AI 是"引擎"（真正撰写报告的是 AI）。这些步骤会带你在 VS Code 中配置 GitHub Copilot，如果你从零开始，这是最快的方式。BugIt 也可以配合 Claude 扩展、任何其他能够运行命令的助手使用，或者使用你自己的 AI 密钥独立运行。如果你已经在用其中之一，可以直接跳到步骤 2，用那个工具而不是 VS Code 打开 BugIt 文件夹。

开始之前：你需要最新版的 VS Code，以及一个能够运行命令的助手，例如 GitHub Copilot Chat 或 Claude 扩展。

它是什么：GitHub Copilot 是 GitHub（微软旗下）出品的 AI 助手。它有免费版供轻度使用，也有付费版供重度使用。如果你选择它，会在步骤 1 中安装它并登录。

1. 你需要一个 **GitHub 账户**。如果还没有，可在 `github.com/signup` 免费注册一个。
2. Copilot 的免费版足够你试用 BugIt。如果你整天都在提交缺陷，付费的 "Copilot Pro" 套餐物有所值，你可以随时在 `github.com/features/copilot` 升级。
3. 你会在步骤 1 中实际在 VS Code 里开启 Copilot，这里暂时不用做任何其他事。

1 安装 VS Code

VS Code 是微软出品的免费应用。它是 BugIt 所在的"窗口"。

1. 打开浏览器，访问 `code.visualstudio.com`。
2. 点击蓝色的大 **Download** 按钮（它会自动识别你的系统）。
3. 打开下载的文件并安装：**接受所有默认选项**，一路点击 Next/Install 即可。
4. 安装完成后打开 VS Code。

在 VS Code 中开启 GitHub Copilot

1. 在 VS Code 最左侧，点击 **Extensions** 图标（看起来像四个小方块）。
2. 在搜索框中输入 **GitHub Copilot**。
3. 点击 "GitHub Copilot" 上的 **Install**（如果有提供，也安装 "GitHub Copilot Chat"）。
4. 系统会提示你**登录 GitHub**。点击它，然后在弹出的浏览器窗口中用你的 GitHub 账户登录。

> **成功的标志：**VS Code 底部会出现一个小的 Copilot 图标，聊天面板也会打开（左侧的对话气泡图标，或按 `Ctrl+Alt+I`）。

2 解压你下载的 BugIt

你购买的产品附带一个 .zip 文件（例如 `bugit-qa-agent.zip`）。你需要先解压它，保持压缩状态是无法运行的。

1. 找到该 .zip 文件（通常在**下载**文件夹中）。
2. **右键点击**它 → 选择 **Extract All...**（Windows）或双击它（Mac）。
3. 选择一个简单的位置，比如你的**文档**文件夹，然后点击 Extract。
4. 现在你会得到一个包含大量文件的 **BugIt** 文件夹。记住它的位置。

> **不要放进 OneDrive 或其他同步文件夹**
>
> 云同步文件夹可能导致文件冲突。放在"文档"或"桌面"里最合适。另外也不要重命名文件夹内的文件，保持原样即可。

3 在 VS Code 中打开 BugIt

1. 在 VS Code 中，点击 **File → Open Folder**（左上角菜单）。
2. 导航到你解压 BugIt 的位置（例如"文档"）。
3. 单击一次 **BugIt** 文件夹以选中它，然后点击 **Select Folder**。
4. 如果 VS Code 询问 "Do you trust the authors of the files in this folder?"，点击 **Yes, I trust the authors**。

> **成功的标志：**你会在 VS Code 左侧看到 BugIt 的文件列表（比如 `tools`、`.github` 等文件夹，以及 `README.md` 等文件）。

4 在聊天中选择 BugIt 代理

1. 打开 Copilot **Chat** 面板：点击左侧的对话气泡图标，或按 `Ctrl+Alt+I`（Windows）/ `Cmd+Alt+I`（Mac）。
2. 聊天框顶部有一个小的**模式下拉菜单**（可能显示 "Ask" 或 "Agent"）。
3. 点击它，从列表中选择 **BugIt QA Agent**。

> **列表中看不到 "BugIt QA Agent"？**
>
> 请确认你打开的是 BugIt 文件夹（步骤 3），而不是其他文件夹，因为代理是从这个文件夹内部加载的。如有需要，关闭并重新打开该文件夹。

5 激活你的许可证

你只需要一个 **BugIt 账户**，无需复制或粘贴任何内容。你在浏览器中批准这台设备，其余由 `Activate` 命令完成。

1. 在聊天框中输入 `hi` 并按回车。
2. 输入 `Activate`；BugIt 会在你的浏览器中打开 BugIt Portal；登录你自己的账户，选择你的 Solo 或 Team 权益，并批准这台设备；然后返回 VS Code，激活会自动完成。你的密码始终留在浏览器中，绝不会输入到 VS Code 里。

> ✅ **成功的标志：**BugIt 会确认激活完成，并告诉你登录使用的是哪个账户。

> **请对你的账户保密**
>
> 你的账户和权益与你本人绑定。请勿分享、公开发布或转卖，被共享的访问权限可能被撤销。Team 成员各自使用自己的账户登录。没有共享的密钥，也没有共享的登录。

6 接受条款（仅需一次）

第一次使用时，BugIt 会展示一份简短摘要，并要求你先接受才能继续。

1. 阅读它展示的简短摘要（你会在每份工单提交前进行审阅；你项目的安全由你自己负责；这些保障措施是辅助手段，而非保证，而且它们由 Copilot 中当前回答你的 AI 模型来执行，较轻量/免费的模型可能偶尔会跳过某一步或需要你重复一条命令，而更强的模型则不会）。
2. 在 BugIt 显示的两个选项中选择 **I agree**。

> ✅ **成功的标志：**BugIt 会确认并继续进行设置。系统只会问你这一次，之后它会记住。

7 运行 "Begin setup"

现在 BugIt 会用大白话向你提问，从而完成自我配置。你永远不需要手动编辑设置文件。

```
你输入
Begin setup
```

它会问一些简短的问题，只开启你选中的功能。其中最重要的是你的跟踪系统。

8 连接你的跟踪系统

设置已经记录了你使用哪一个跟踪系统。这一步才是让 BugIt 真正能向它提交工单的步骤：你在自己的账户中创建一个 API 令牌，再用一条命令把它保存到这台电脑上。BugIt 会带着该令牌直接通过跟踪系统自己的 REST API 提交工单，因此不需要让任何东西保持运行，提交工单也从不使用 MCP 服务器。

8a · 在你自己的账户中创建令牌

你的跟踪系统	在哪里创建
Jira Cloud	`id.atlassian.com/manage-profile/security/api-tokens`
Azure DevOps	`dev.azure.com/YOUR-ORG/_usersSettings/tokens`
GitHub Issues	`github.com/settings/personal-access-tokens/new`
GitLab Issues	`gitlab.com/-/user_settings/personal_access_tokens`
Bugzilla	你自己的 Bugzilla：Preferences → API Keys
YouTrack	你自己的 YouTrack：头像 → Profile → Account Security → Authentication
Linear	`linear.app/settings/account/security`
Shortcut	`app.shortcut.com/settings/account/api-tokens`
ClickUp	`app.clickup.com/settings/apps`
Asana	`app.asana.com/0/my-apps`
Trello	`trello.com/apps/admin` (一个 API 密钥，外加一个用它授权得到的令牌)

> **令牌一显示就立刻复制**
>
> 大多数服务的新令牌只会显示一次。立刻复制它，并妥善保存，直到 BugIt 替你保存好（步骤 9）。选择你跟踪系统提供的最长有效期，这样就不必很快再重来一遍。

8b · 保存到这台电脑

可以让助手连接你所用的跟踪系统，它会替你执行。或者在 VS Code 中打开终端，运行你所用跟踪系统对应的命令，把 `jira` 换成 `ado`、`github`、`gitlab` 等即可。

你运行（在终端中）

```
python tools/connect.py jira
```

8c · 把令牌粘贴到掩码提示中

该命令会在一个本地掩码提示中索取令牌：它不会显示在屏幕上，也绝不会进入聊天、工单或 `config.json`。只把它粘贴到那里，不要粘贴到别处。随后 BugIt 会登录，列出该令牌实际可见的项目，读取你被允许创建的工单类型，并**拒绝保存无法提交的凭据**。因此设置能够完成，就意味着提交可用。

8d · 确认已连接

想随时确认，可在终端运行检查，或直接在聊天框中询问：

你运行（在终端中）

```
python tools/connect.py jira --check
```

你输入

```
Check status
```

成功的标志：BugIt 会把你的跟踪系统列为已连接/健康状态。

关闭 VS Code 后无需再启动任何东西

你的 API 令牌保存在操作系统的凭据存储中，而不是 VS Code 里。所以重新打开 VS Code 时没有任何东西需要启动，BugIt 可以立即提交。想确认的话，随时运行 `python tools/connect.py jira --check`。

9 保存你的登录信息

当你在步骤 8c 中保存令牌后，你的电脑会自己保护这份凭据：在 Windows 上是一条只有你这台机器上的 Windows 账户才能打开的加密记录，在 macOS 和 Linux 上则存放在钥匙串或密钥服务中。绝不以明文文件保存，除了你配置的那个跟踪系统之外也绝不发送到任何地方。想确认哪些跟踪系统已保存凭据，输入：

你输入

```
Check saved credentials
```

……BugIt 会显示哪些跟踪系统已保存凭据，但从不显示凭据值。BugIt 从不显示或复制你保存的令牌值。

---

10 确认你已就绪

在正式开始提交之前，让 BugIt 帮你再检查一遍：

你输入

```
Check readiness
```

它会列出仍缺少的内容（通常只是"连接你的跟踪系统"或"激活"）。当一切都显示绿色时，你会看到 "✅ Setup complete."

设置到此完成，一劳永逸

你不会再重复第一部分。从现在起，打开 BugIt 只需：打开文件夹 → 输入 `hi`。之后想更改任何选项，直接输入 `Begin setup` 并说明要改什么即可。

没有 GitHub Copilot？还有终端向导

如果你无法使用 Copilot，打开内置终端（Terminal → New Terminal）并运行 `python tools/setup_wizard.py`。它会问几个问题，并在没有 AI 的情况下替你写好设置。你也可以使用自己的 AI 密钥独立运行 BugIt，见常见问题。

第二部分 · 日常使用

设置完成后，你完全可以只在聊天中工作。可以自然地描述，也可以使用下面的确切示例。随时输入 `help` 查看完整列表。

撰写缺陷报告

这是 BugIt 的主要工作。用一句大白话描述缺陷；它会撰写完整工单，自动填写版本和类别，检查重复项，展示预览，并且只有在你以整条消息回复 `FILE IT` 之后才会提交。仅仅说“好”是不够的。

你输入

```
Write a bug report: 在iPhone上点击保存时应用每次都会崩溃
```

它会起草标题、重现步骤、预期与实际结果、严重程度和平台，并会把你确认的严重程度记录在工单正文中。如果跟踪系统及项目支持原生的优先级或严重程度字段，BugIt 也会设置该字段。GitHub、GitLab 和 Trello 没有这样的字段。想在提交前做些修改？直接说出来即可，例如"把严重程度改为高"或"补充说明是从最新版本开始出现的"。

快速缺陷（快速提交）

对于常见类型（crash, error, missing, layout, slow, stuck, data, visual, audio, blocker），快速表单会跳过一些问题，并替你填好类别和优先级，并且会跳过重复检查以省去一次往返。当你已经确定这个问题是新的时候再使用它；预览会这样说明，而标准报告仍会运行完整的重复检查。

你输入（示例）

```
Quick bug: 更新后启动崩溃
Quick bug: layout 设置界面的按钮和文字重叠了
Quick bug: blocker 支付步骤卡住无法继续
Quick bug: slow 仪表盘要花 20 秒才能加载完
```

崩溃缺陷报告

如果你连接了崩溃工具（如 Sentry），BugIt 会完成上述所有工作，并额外把你的报告与崩溃匹配，替你提取堆栈跟踪信息。

你输入

```
Write a crash bug report: 打开地图时游戏崩溃
```

验证与关闭评论

在你测试完修复后，BugIt 会撰写一条整洁的评论供你发布到工单上。它只撰写（并可选择发布）评论，不会更改工单的状态。你仍需自己在跟踪系统中关闭/解决/重新打开该工单。

你输入	你会得到
`Close #123`	询问是哪种场景（修复已确认、按要求关闭、按设计如此、重新打开……），然后撰写对应的评论。
`Write a verification comment for the build you tested on Windows`	一条"修复已确认"的评论，并填好构建版本详情。
`Write a reopen comment for the build you tested on Windows`	一条"问题仍然存在"的评论（之后你自己重新打开该工单）。

翻译

使用你团队的确切措辞（来自你的术语表，见第三部分），在你已配置的语言之间翻译报告或术语。

你输入（示例）

```
Translate to ja: 个人资料页面上的保存按钮没有反应
Translate this bug report to English: [paste text]
```

查阅内容（规格文档与术语表）

如果你连接了规格文档或填写了术语表，可以向 BugIt 询问有关你项目的问题。

```
你输入（示例）
What is [your term]?
Walk me through the checkout flow
What's new in specs?
```

重复检测

在标准报告中，这会在写入任何内容之前自动发生：BugIt 会在你的跟踪系统中搜索匹配的工单（即使措辞略有不同）并在写入任何内容之前提醒你，由你来决定。这是一次检查而不是保证：搜索读取的工单数量有上限，没有被找到的重复项仍然可以提交。快速表单会特意跳过这次搜索以省去一次往返，并在预览中说明这一点，因此你始终知道自己用的是哪一种。

强力功能（如果你已开启）

输入这个	它的作用
`Triage digest`	即时站会摘要：按区域/严重程度分类的未解决缺陷、最旧的标记项。
`Export bug to file`	将报告保存为 Markdown/CSV/JSON 而不提交（在跟踪系统尚未设置好之前非常好用）。
`Undo last filing`	显示审计日志中最近一次写入。BugIt 无法删除或关闭工单；在重新预览并输入 `FILE IT` 之后，它只能删除自己创建的评论或附件。
（自动）	紧急缺陷的 SLA 标记 · 分类专属字段。

完整命令列表

```
你输入
help
```

……随时显示代理内的每一条命令。

> **随时安全练习**
>
> 输入 `dry run`，BugIt 会撰写整份报告但**不会**提交任何内容，非常适合学习或测试。

命令速查

你随时可以用自己的话描述想做的事。如果你更喜欢固定说法，下面是 BugIt 能识别的准确表述。

你输入	会发生什么
提交缺陷	
`Quick bug: [type] [subject]`	快速提交。会有意跳过重复检索，并在预览中说明这一点。
`dry run`	写出完整报告但不提交任何内容。安全的练习方式。
`FILE IT`	唯一会真正提交的输入。作为整条消息输入一次，只对你刚读过的草稿有效。
`Export bug to file`	把报告保存为文件，而不是发送到你的跟踪系统。
`Undo last filing`	显示最近一次写入，并可移除 BugIt 添加的评论或附件。它无法关闭工单。需要开启审计日志。
`Triage digest`	你近期提交内容的汇总。
你项目的知识	
`What is [term]?`	在术语表中查一个术语。
`What's new in specs?`	你指给 BugIt 的规格文件夹中的最近改动。
`Look up crash [ID]`	搜索你的崩溃工具，只返回链接。不会写报告。
`Look up #[ticket]`	搜索你的跟踪系统，只返回链接。不会写报告。
`Seed glossary: [text]`	根据一段文本给出术语表候选。每一条都由你确认。
`Learn from my recent tickets`	从跟踪系统中已有的缺陷里学习你的写作风格。
设置与健康检查	
`Begin setup`	完整的首次设置，或之后只改其中一项。
`Check status`	我的工具连上了吗？
`Check readiness`	距离能够提交还差什么？
`Add a custom MCP server`	连接一个未内置的工具。
`Fix servers`	诊断并修复 MCP 连接。
`Check saved credentials`	只显示名称。BugIt 从不显示已保存凭据的值。
`Talk to me in [language]`	更改 BugIt 在这台机器上与你对话所用的语言。这与撰写报告的语言是分开的。

你输入	会发生什么
`help`	在聊天中显示这张表的简版。
许可证、更新与备份	
`Activate`	在浏览器中授权这台机器。绝不会把密钥或密码粘贴到编辑器里。
`License status`	你的方案、权益和离线状态。
`What version am I on?`	你的版本，以及是否有更新在等待。
`Update`	就地安装最新的已签名版本。你的设置会保留，并且会先备份。
`Back up my settings`	在本地保存配置、术语表、写作风格和 MCP 设置。不包含凭据。
`List backups`	查看可以回退到的备份。
`Restore my settings`	回退到其中一个。
`Create support bundle`	一个已脱敏的诊断文件，可随支持邮件一起发送。不含凭据、工单和路径。
Team	
`Authorize this Team device`	在浏览器中用你自己的 BugIt 账户批准这台机器。
`Manage my devices`	打开 BugIt 门户，在那里登出设备并释放它的席位。
`Manage my Team`	打开 BugIt 门户，在那里管理成员、角色、席位和许可证。

第三部分 · 打造专属设置：添加你项目的知识

开箱即用时，BugIt 是一个白板式的 QA 代理。以下是购买后教它了解你项目的方法。不需要写代码，大多只是在聊天中和它对话。你添加的一切都只保存在你的电脑上。

1 · 你团队的用语（术语表）

这能让翻译和类别推测对你的项目更加准确。

1. 在文件列表中，打开 `.github/glossary/terms.template.md`。
2. 在表格中填入你团队的用语：功能名称、界面名称、条目名称、缩写，以及（如果你使用两种语言）它们的翻译。
3. 保存文件（`Ctrl+S`）。就这样，BugIt 从此会使用这些确切的术语。

快捷方式

在 `Begin setup` 期间打开"术语表自动填充"，BugIt 会从你已有的工单中建议术语，你只需逐一确认。

2 · 你的缺陷风格（团队风格）

想让工单完全匹配你团队的格式（标题风格、严重程度标签、必填字段）？你不需要描述它，只需**展示**给它看：

1. 在聊天中说："match my team's style."
2. 粘贴你跟踪系统中**一张**现有工单的截图。
3. 图片由已在你聊天中的 AI 模型按其自身提供商的条款读取，BugIt 会尽力屏蔽提取出的文本中常见的个人数据，保存你的格式，然后在之后每一份工单上都遵循它。

3 · 你的类别、平台与字段

只需用大白话告诉 BugIt，它就会替你更新设置：

你输入（示例）

```
My categories are Gameplay, UI, Audio, and Networking
We test on Windows and Xbox
```

4 · 你自己的工具（自定义连接）

使用的工具不在内置列表中？像连接内置工具一样连接它：

你输入

```
Add a custom MCP server
```

给它起一个简短的名称并提供地址（必须是 `https://`）；BugIt 会替你连接并进行健康检查。

5 · 随手纠正即可

最简单的方法：当 BugIt 起草工单时，直接修正你不喜欢的地方，比如一个标签、措辞或严重程度。开启"从你的修改中学习"（推荐）后，它会记住你的偏好并在下次应用。你的项目知识会在你的电脑上自然而然地积累起来，而且其中没有任何内容会发送给 Taskivator。

你添加的一切都属于你

你的术语表、团队风格、类别和学习到的偏好都保存在你的电脑上，并在每次更新前备份。当 BugIt 执行你的请求时，相关内容可能会发送给你所连接的 AI 模型和工具。这些内容都不会发送给 Taskivator；共享包只有在你选择发送时才会离开你的电脑。

6 · 与团队共享你的设置（团队许可证）

一旦你按自己的想法设置好了术语表、团队风格和跟踪系统选择，就可以把这套设置分享给团队其他成员，而不必在每台机器上重复步骤 1-3：

```
你运行（设置调整满意后运行一次）
python tools/share.py export
```

这会生成 `config.share.zip`，把你的跟踪系统/崩溃工具/通讯选择、术语表和团队风格打包在一起，不含任何密码或令牌。把它发给你的团队。

```
每位团队成员先运行这一步，用于审阅
python tools/share.py import config.share.zip
```

这一步只显示将会发生的更改，不会写入任何内容。

```
审阅之后，应用设置
python tools/share.py import config.share.zip --apply
```

这会应用跟踪系统、崩溃工具和通讯设置，不用重新输入类别。如果还要采用术语表和团队风格，必须先审阅这些内容，再改用 `python tools/share.py import config.share.zip --apply --apply-instructions`。他们自己的许可证和设备设置不受影响，凭据也不会随包传递。

> **如果导入更改了数据发送的位置**
>
> 如果共享的设置会新增一个自定义连接或通知地址，预览会在应用任何内容之前准确展示将会发生的变化。应用时，BugIt 也会先为该队友当前的设置生成快照，因此 `Restore my settings` 可以在结果不符合预期时撤销。

第四部分 · 更新、备份与安全

获取更新（一条命令）

有新版本可用时，BugIt 会在你输入 `hi` 时提及一次。只要你的 BugIt 许可证有效，软件更新即包含在内。安装方法：

```
你输入
Update
```

1. BugIt 会先为你的文件做一次全新备份。
2. 它会下载新版本，并检查其真实性和完整性（更新经过加密签名，伪造或被篡改的更新会被拒绝）。

3. 它会在不改动你的设置、术语表、团队风格、许可证或令牌的情况下完成安装。
4. 它会要求你重新加载 VS Code（`Ctrl+Shift+P` → 输入 "Reload Window" → 回车）。开始一个新的聊天，你就用上了新版本。

> **永远不需要删除文件夹**
>
> 与首次安装不同，更新是原地进行的。你永远不需要删除或重新下载任何东西，你的设置也始终会被保留并备份。

哪些可以放心手动编辑，哪些不行

安全，且每次更新都会保留： `config.json`、`.github/instructions/house-style.instructions.md`（见第三部分，这是自定义行为的官方支持方式）、`.github/glossary/terms.template.md`、`.github/instructions/learned.instructions.md`，以及 `.vscode/mcp.json`。

> **不安全：请勿手动编辑这些文件**
>
> 代理文件（`.github/agents/bugit-qa-agent.agent.md`）、`.github/instructions/` 下的任何其他文件，以及 `tools/` 中的所有内容，都是产品本身，而非你的设置。更新会**直接静默覆盖**它们，而且和上面的文件不同，**没有备份可以恢复它们**。BugIt 会在启动时以及安装更新前再次检查这一点，如果发现有改动会提醒你，但最保险的做法就是干脆不要编辑它们。

备份与恢复

你输入	它的作用
`Back up my settings`	保存你的配置、术语表、团队风格、MCP 端点和学习数据的本地快照；你的已签名设备权益和凭据值不包含在内。
`List backups`	显示每一份已保存的快照及其日期。
`Restore my settings`	回滚到某个快照（并先为你当前的状态生成快照，所以恢复操作本身也是可撤销的）。

> **每次更新前自动进行**
>
> BugIt 总是会在安装新版本前先备份，因此更新绝不会丢失你的设置。如果更新后你的配置出现异常，`Restore my settings` 即可修正。

安全：受保护的内容

未经你同意，不做任何事

每条工单和评论都会预览；不可逆的操作需要你输入 `FILE IT`；仅输入"是"不会执行。

Dry-run = 零写入

说 `dry run`，就完全不会连接你的跟踪系统，读取也包括在内。dry run会抑制所有对外的工单、评论、编辑、附件、删除和通知（备份或导出等你主动运行的本地命令仍可能创建本地文件），不会搜索，不会做重复检查，也不会做连接检查，因为每一项同样会连到你的看板并解锁你保存的令牌。终端里的 `QA_AGENT_DRY_RUN=1` 才是随附命令在代码中强制执行的开关；在聊天里说"dry run"只是请助手先别动手，这有用，但不是同一种保证。

文件中不含密钥

令牌保存在你操作系统的保险库中，绝不以明文文件保存，也绝不发送给我们。

运行在你自己的电脑上

你的工作内容只会发送给你连接的工具和你自己的 AI 模型。

你的数据与隐私

BugIt 发送给 Taskivator 的唯一内容是许可证/更新数据：你在浏览器中、在 BugIt Portal 上完成的 BugIt 账户登录，一个单向哈希的设备 ID，一个随机的安装标识符，你的设备名称（主机名）和操作系统名称，BugIt 版本和你选择的套餐筛选，短时有效的加密质询数据、浏览器打开的 Portal 链接中的一次性批准令牌、每次激活生成的确认密钥（其哈希；仅在撤销这台设备的访问时才发送密钥本身），以及你为这台设备批准的 Solo 或 Team 权益。这里没有任何激活码；你的密码始终留在浏览器中。你的缺陷报告、规格文档、术语表、设置和令牌绝不会发送给 Taskivator。你提交的内容会去往你发送的地方：你自己账户中的跟踪系统，用你自己的凭据写入。如果你使用托管的 AI 模型，草稿也会到达该模型的提供方。没有遥测，没有分析，绝不会有。完整详情见你 BugIt 文件夹中的 `PRIVACY.md` 文件。

风险与重要注意事项：请务必阅读

AI 可能出错

BugIt 使用 AI 模型起草报告，它可能误读细节或凭空捏造内容。在你确认（输入 `FILE IT`）之前，请务必阅读预览内容。你批准的才是最终提交的内容。

它会提交到你真实的跟踪系统

一旦你确认，就会创建一个真实的工单。想练习？用 `dry run` 撰写报告而不提交。

你项目的安全由你负责

你如何使用 BugIt，以及你项目、数据和跟踪系统的安全，都是你自己的责任。这些保障措施能降低风险，但不能替代你的审阅。

BugIt 目前做不到的事

BugIt 会向它列出的全部十一个跟踪系统提交，每一个都使用你在自己账户中创建的凭据；保存之前，BugIt 会对着你选定的提交目标核验这份凭据。能进入可排序优先级字段的内容因系统而异，GitHub、GitLab 和 Trello 根本没有这样的字段，所以那里的严重程度写在工单正文中。Confluence、Sentry 和 Notion 是只读的；只有它们才需要你自己的兼容 MCP 服务器，由你在 BugIt 的协助下完成设置。Slack、Teams、Discord 和电子邮件通知是内置的。它使用你自己的 AI（Copilot 或你自己的 OpenAI/Anthropic 密钥），不自带模型；脱敏是尽力而为；运行环境为 VS Code、Claude 扩展和普通终端。一个许可证 = 一台设备（Solo），或 5 个成员席位、每人一台设备（Team）；一个团队可以持有多个 Team 许可证，席位相加。结果与一致性还取决于回答的是哪个 AI 模型：更强的模型比非常轻量或免费的模型更可靠地遵循安全步骤。

第五部分 · 故障排查与常见问题

出现问题时，先用大白话向助手描述一遍。设置和日常使用中的大多数问题都可以直接在编辑器里诊断。

先试试这个：让 AI 帮你修复

无论出现什么问题（无论是在设置期间还是日常使用中），你最快的解决办法几乎总是**直接在聊天中问助手**。用大白话描述发生了什么（例如“设置进行到一半就停了”或“我的跟踪系统连不上”），然后让它帮你修复。AI 能够当场诊断并修复大多数问题。在给我们发邮件之前，请先试试这个办法，它能在几秒钟内解决绝大多数问题。

应用 AI 的修复：那个 "Keep" 按钮

如果助手通过修改文件来修复问题，VS Code 会显示一个小的 Keep / Undo 提示条。如果是你要求它修复的，并且你对结果满意，点击 **Keep**，这个改动只会应用到你自己电脑上的这份副本。点击 **Undo** 则丢弃它。你 Keep 下来的内容不会与任何人分享。

你看到的	该怎么做
"Activate to start"	输入 `Activate`；BugIt Portal 会在你的浏览器中打开。登录，选择你的 Solo 或 Team 权益，批准这台设备。还没有账户？到商店购买一个。
浏览器没有打开，或激活没有完成	再次输入 `Activate` 以重新打开 BugIt Portal，然后完成登录并批准这台设备。请让该命令一直运行，直到它提示授权已保存在这台电脑上。无需粘贴任何密钥。
"No tracker enabled"	输入 `Begin setup` 并选择你的跟踪系统。
提交被拒绝	运行 `python tools/connect.py jira --check`。它会告诉你跟踪系统认为你是谁、BugIt 可以创建什么，并在 `fixes` 中列出具体的解决办法。

你看到的	该怎么做
它一直要求你激活	输入 `Activate`，并在浏览器中完成登录。BugIt 从不显示已保存的凭据值。（第一次遇到？见步骤 8c。）
它无法提交缺陷	输入 `Check readiness`，它会准确列出缺少的内容（通常是尚未保存跟踪系统的令牌）。
更新后看起来有问题	如果你的配置被改动了，输入 `Restore my settings`。如果 BugIt 自身的文件损坏，输入 `Repair my BugIt install`，并先查看预览。如果文件完好，但新版本仍然运行异常，请发邮件至 support@bugit.dev。
代理看起来"偏离脚本"	输入 `Read and follow all your given instructions`，或开始一个新的聊天。这种情况在轻量/免费的 AI 模型上更容易出现，在 Copilot 的模型选择器中换一个更强的模型会更稳定。
回答感觉很笼统/不针对你的项目	提供更多上下文，或在 `Begin setup` 期间给它看一张现有工单，让它学习你的风格。重新运行 `Begin setup` 以进一步优化。
你想更改某项设置选择	输入 `Begin setup` 并说明要改什么（例如 "add a 跟踪系统" 或 "start over"），它只会重新打开对应部分。如果设置只是中途被打断，单独输入 `Begin setup` 会从中断处继续，而不是重新开始。

> **内置健康检查**
>
> `Check status`（我的工具都连接好了吗？）· `Check readiness`（我还差什么才能提交？）。想要更深入的检查，打开 Terminal → New Terminal 并运行 `python tools/selftest.py`。

常见问题

我需要懂编程吗？

不需要。如果你会安装应用、会打字造句，你就能使用 BugIt。技术部分它都替你完成了。

它会在我不知情的情况下提交缺陷吗？

绝不会。每条工单和评论都会先预览，并且不可逆的操作需要你输入 `FILE IT`；仅输入"是"不会执行。用 `dry run` 可以零写入地练习。

我每次都要启动服务器吗？

不需要。提交使用的是你用 `python tools/connect.py` 保存的 API 令牌，所以没有任何东西需要启动。只有 Confluence、Sentry、Notion 这类可选的只读集成才需要 MCP 服务器。

我可以在两台电脑上使用它吗？

Solo 版一次只能用一台。在 Team 中，每位成员使用自己的账户和自己的设备，席位数量来自这个团队持有的 Team 许可证：每个 Team 许可证增加 5 个席位。更换到新设备没问题：激活新设备后，旧设备会立即失去续期能力。其名额要等到旧设备重新联网（确认已交出访问权限），或其当前的 72 小时离线许可证到期后才会释放，因此没有固定的等待时间。输入 `License status` 即可查看。

如果许可证服务器宕机了怎么办？

你可以继续工作。成功完成在线激活后，缓存的许可证可让此设备离线工作最多 72 小时，之后需要在线验证。因此在这段时间内，一次服务中断，或只是没有网络的周末，都不会打断你工作。

我需要 GitHub Copilot 吗？

不需要。BugIt 可以在任何能够运行命令的助手中运行：Copilot Chat、VS Code 中的 Claude 扩展，或普通终端。Copilot 只是最简单的方式。如果你没有它，可以在终端中运行 `python tools/setup_wizard.py` 完成设置，BugIt 也可以搭配你自己的 OpenAI/Anthropic 密钥独立运行（`python standalone/run.py`）。

我的数据是私密的吗？

是的。Taskivator 只接收许可证/更新数据：在浏览器 Portal 中登录你的 BugIt 账户、一个单向哈希的设备 ID、一个随机的安装标识符、你的设备名称（主机名）和操作系统名称、BugIt 版本和你选择的套餐筛选、短时有效的加密质询数据、浏览器打开的 Portal 链接中的一次性批准令牌、每次激活生成的确认密钥（其哈希；仅在撤销这台设备的访问时才发送密钥本身），以及你为这台设备批准的 Solo 或 Team 权益。BugIt 软件不使用 Google Ads 衡量，也不发送产品遥测数据。这里没有任何激活码；你的密码始终留在浏览器中。你的工单本身只会发送给你连接的 AI 模型和跟踪系统，绝不会发给我们。

我可以编辑 BugIt 的文件吗？

你应该自定义属于你的部分：术语表、团队风格和设置（第三部分）。只是不要禁用授权/激活功能。如果你在工具本身发现了真正的缺陷，请发邮件联系支持，以便在下一次更新中为所有人修复它。

它支持哪些缺陷跟踪系统？

十一个，而且 BugIt 会向其中每一个提交：Jira Cloud、Azure DevOps、GitHub Issues、GitLab Issues、Bugzilla、YouTrack、Linear、Shortcut、ClickUp、Asana 和 Trello。每一个都有引导式设置，每一个都使用你在自己账户中创建的凭据写入；保存之前，BugIt 会对着你的提交目标核验这份凭据。差别在于附带能力：BugIt 不会向 GitHub、GitLab 或 Bugzilla 上传文件，并会在你尝试之前告知（GitHub 根本没有可用凭据认证的上传，GitLab 和 Bugzilla 有，但 BugIt 尚未认证），ClickUp 收得下文件却删不掉，而 GitHub、GitLab 和 Trello 没有优先级字段，所以那里的严重程度写在工单正文中。在设置时选择你的那个，随时可用 `Begin setup` 切换。

提交前它会发现重复缺陷吗？

在标准报告中会。在写入任何内容之前，它会在你的跟踪系统中搜索相似的报告（即使措辞略有不同），并把匹配项展示给你，让你避免重复提交同一个缺陷。是否继续仍由你决定。快速表单会特意跳过这次搜索以省去一次往返，并在预览中告诉你。

它能用不止一种语言撰写工单吗？

可以。在设置时添加第二语言，之后每份报告都会用两种语言撰写，分成独立的段落，并使用你的术语表确保产品术语准确无误。它绝不会随意发挥翻译。

我可以附加截图吗？

请让 BugIt 来附加。它会创建一个证据文件夹并打开它，同时告诉你确切路径；把文件放进去，然后告诉它你已完成。BugIt 会检查文本文件中是否含有凭据，若发现则停下而不附加，并在发送前向你展示预览。它无法看到图片内部，因此截图请你自己先确认再批准。附加会在收到专门的 `FILE IT` 后进行。粘贴到聊天中的文件 BugIt 无法读取，因此它始终为你提供该文件夹。

它会在我的报告中隐藏密码和令牌吗？

开启脱敏功能后，它会在提交前尽力屏蔽每份草稿中的邮箱、令牌和 IP 地址。你在预览中始终能看到完整的文本。

我不小心提交了一个工单，能撤销吗？

预览和输入 `FILE IT` 的确认步骤能阻止大多数误操作发生在写入之前。如果你开启了审计日志，输入 `Undo last filing`：它会显示 BugIt 写入了什么，并可撤回 BugIt 自己添加的评论或附件。关闭或删除工单始终是你在跟踪系统中亲自完成的操作，因为没有任何跟踪系统为 BugIt 提供这个途径。

它能帮我验证或关闭一个缺陷吗？

可以。请求一条验证评论，它会针对现有工单起草一条清晰的通过/不通过说明（使用你的语言），你确认后它才会发布。

我可以把它用于不止一个项目吗？

每个 BugIt 文件夹只针对一个项目的跟踪系统进行设置。对于另一个项目，要么重新运行 `Begin setup` 指向新的位置，要么为每个项目保留一份独立的副本。

我如何获取更新？

有新版本发布时，BugIt 会在你开始聊天时提及。输入 `update`，它会原地安装：你的术语表、团队风格和设置都会被保留，并且会先进行备份。

我的许可证到期后会怎样？

期限结束后，请从你的账户续费；你的设备会在下一次在线签入时应用续费后的权益。72 小时的离线租约是另一回事，它只针对许可证服务器的临时中断，而非到期的期限。随时输入 `License status` 查看。

如果我续费或再购买一次，天数会累加吗？

天数永远不会累加。续费会**替换**你当前的权益；期限重新开始计算，未用完的天数不会结转，所以请在当前期限快结束时再续费。**席位则不同。**在 Team 中，第二个 Team 许可证会把它的 5 个席位加到原有席位上，各自按自己的期限计算，所以两个许可证就是 10 个席位。请看下面关于 Team 的问题。

Team 许可证：常见问题

本页的所有操作都在浏览器中的 **BugIt 门户**的 **Team** 页面完成，而不是在编辑器里。如果你买的是 Solo 许可证，可以跳过本页。

Team 许可证能得到什么？

5 个席位，为期一年，非订阅，不会自动续费。团队中每个人用自己的 BugIt 账户登录，并在自己的设备上激活 BugIt：没有共享密钥，也没有共享登录。成员、邀请、角色和席位都在门户的 Team 页面管理。

怎么添加成员？

打开 Team 页面，向对方的电子邮件地址发送邀请；你可以一次粘贴多个地址。每个邀请只对发送到的那个地址有效，所以转发链接对别人没有用。待处理的邀请在被接受或取消之前就占用一个席位，因此团队可能在所有人加入之前就已满。

谁能做什么？

所有者可以做任何事，包括账单和把团队交给别人。**管理员**可以邀请和移除成员，并修改团队的共享默认值。**成员**使用 BugIt，可以随时离开。所有者不能被移除，只能转让。

我们不止五个人。可以再买一个 Team 许可证吗？

可以。购买第二个 Team 许可证，然后在 Team 页面把它加到同一个团队。**席位相加：**两个许可证是 10 个，三个是 15 个。没有固定上限，因为每个席位都来自你拥有的许可证。添加后立即生效。

如果我加了第二个 Team 许可证，我的团队什么时候到期？

每个许可证保留自己的到期日，并在此之前支付自己的 5 个席位。只要**任意一个**许可证仍然有效，团队就继续可用，所以团队的到期日是两者中较晚的一个。较早的那个到期后，团队会失去它所支付的 5 个席位，但团队本身不会关闭。第二个许可证不会延长第一个：如果一个还剩三个月，你又加了一个全新的一年期许可证，那么三个月内是 10 个席位，之后九个月是 5 个席位。续费较早到期的那个，就能保住全部 10 个。Team 页面会显示每个许可证及其各自的到期日，你随时都能看清哪个先到期。

如果人数超过席位会怎样？

我们会在许可证到期前 30 天、7 天和 1 天写信给团队，所以绝不会突然发生。如果许可证确实在团队占用席位超过已付席位时到期，会先由待处理的邀请交出席位，之后由最近加入的人失去访问权限。所有者永远不会被移除，因此总有人可以续费。已经提交的内容不会被删除，停止的只是对 BugIt 的访问。

如果拥有团队的那个人离职了怎么办？

所有者可以在 Team 页面提名一位**继任者**。如果之后联系不上所有者，继任者可以申请接管；团队所有人都会收到通知，除非所有者在此之前叫停，否则 7 天后生效。联系得上的所有者可以随时直接转让团队，转让前会要求用双重验证确认身份。

整个团队能用同一套设置吗？

可以。把 BugIt 按你的想法设置好，运行 `python tools/share.py export`，每位同事对你发送的文件运行 `import`（第三部分第 6 节）。Team 页面还会记录团队商定的跟踪系统、项目、指派人和标签，方便所有人在一处查看。**绝不要在那里填写令牌或密码：**团队里每个人都能读到这些设置。真正的凭据始终由每个人在自己的机器上用 `python tools/connect.py` 单独保存。

有人离开了团队。他的席位怎么办？

在 Team 页面移除他：他的设备会被登出，席位立刻可以用于你的下一次邀请。成员也可以自己离开。无论哪种情况，他自己的 BugIt 账户仍然属于他，结束的只是对你团队的访问。

收据和续费通知会发到哪里？

默认发到所有者账户的地址。所有者可以在 Team 页面填写公司名称和单独的账单邮箱，该团队的通知和收据就会发到那里。这不会改变团队归谁所有。

第六部分 · 许可证与支持

许可条款（通俗摘要）

完整的、具有约束力的条款在你 BugIt 文件夹中的 `LICENSE` 文件里，以该文件为准。简而言之：

主题	通俗说明
授权使用，而非出售	你购买的是使用 BugIt 的许可，而非所有权。Taskivator 保留全部权利。
你的席位	Solo = 同时 1 台设备。Team = 每个 Team 许可证 5 个成员席位，每人一台设备，同一个团队可以由多个 Team 许可证共同支付。你的账户和权益是你个人专属的，请勿分享、转卖或公开发布。
撤销	被共享、退款、拒付或滥用的访问权限可能被撤销。
可以自定义，但……	可以自由编辑你的配置、术语表和模板，但不要禁用或绕过授权/激活机制。
你的责任	你如何使用 BugIt，以及你项目的安全，都是你自己的责任。每份报告提交前都由你审阅并批准。这些保障措施是辅助手段，而非保证。
"按现状提供"，责任限制	按现状提供，不附带任何担保。在法律允许的范围内，责任上限为你支付的金额，不包括间接或后果性损失。
期限与适用法律	为期一年的许可证，自你购买当天起算（非订阅，不自动续订），受日本法律管辖。退款通过你购买的商店办理。

主题	通俗说明
天数不累加，席位会累加	续费会替换你当前的权益：其有效期从购买时重新开始计算，未用完的天数不会结转，因此请在当前期限快结束时再续费。在 Team 中，每增加一个 Team 许可证，就按它自己的期限增加相应席位；团队会一直可用，直到最后一个有效许可证到期。

你文件夹中的两份文件

`LICENSE`（完整条款）和 `PRIVACY.md`（隐私声明）。激活并使用 BugIt 即表示你接受 LICENSE。

获取帮助

请先查看上面的故障排查和常见问题，它们几乎覆盖了所有情况。然后，按以下顺序：

1 · 让 AI 帮你修复

你最好的第一步：在聊天中描述问题，并让助手帮你修复。如果它修改了文件并且看起来没问题，点击 Keep 应用它（仅限你自己的电脑）。

2 · 运行健康检查

`Check status` · `Check readiness` · 或在终端中运行 `python tools/selftest.py`。

3 · 恢复

`Restore my settings` 可找回你的配置、术语表和团队风格，但不会撤销更新。BugIt 文件损坏时：`Repair my BugIt install`。

4 · 联系真人

只有在指南和 AI 都无法帮到你时：访问 bugit.dev 或发邮件至 support@bugit.dev（技术支持仅提供英文）。购买后 7 天内遇到设置问题？我们会帮你把它跑起来，或协助你通过商店办理退款。

请勿在支持邮件中包含机密的项目信息

每个项目都不一样，你的很多工作内容可能是机密的。如果你确实要给我们发邮件，请只用一般性的语言描述 BugIt 的问题，不要粘贴机密代码、缺陷详情、规格文档、截图或来自你项目的数据。对于发送给我们的任何机密信息，Taskivator 概不负责，我们收到的任何机密内容都会被立即删除。请尽可能先让 AI 助手帮忙，它可以直接在你的编辑器中解决大多数问题。

感谢你选择 BugIt。

提交规范的工单，跳过重复项，把时间还给自己，一次一个缺陷。

BugIt QA Agent · 设置与用户指南 · © 2026 Taskivator. All Rights Reserved. · bugit.dev · 你下载内容中包含的 LICENSE 和 PRIVACY.md 文件为具有约束力的文件。